# Reviewer Pack — Minimal Automorphism Group Order and Communication Complexit…

Entry 4f9191200ded · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 20:34:14 UTC

## Minimal Automorphism Group Order and Communication Complexity Rank Variance

**Entry ID**: `4f9191200ded` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 20:34:14 UTC

### 1. Conjecture

**Field A** (mathematical branch): Group Theory (Automorphism Groups) **Field B** (complexity object): Communication Complexity (Matrix Rank)

**Statement**:

> For all d-regular graphs G, the order of the minimal automorphism group A_G is linearly related to the communication complexity rank variance σ(G), such that σ(G) = O(|A_G|^α) for some constant α ∈ [0,1].

**Rationale (proposer's reasoning)**:

> Automorphism groups capture symmetries of a graph and could potentially influence the structure of the communication complexity. A smaller automorphism group might indicate fewer possible configurations for the communication process, leading to a lower rank variance.

**Taxonomy category**: `GROUP_COMMUNICATION_COMPLEXITY` (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `073d270a425d0bbb`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> For a given d-regular graph G with n ≤ 40 vertices, if the correlation coefficient r^2 between |A_G| and σ(G) is greater than or equal to 0.9, then support the conjecture; otherwise falsify it.

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |

## 4. Novelty audit

**Verdict**: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - `"minimal automorphism group order" AND "communication complexity rank variance"` - `"d-regular graph automorphism group" AND "matrix rank communication complexity"` - `"σ(G) = O(|A_G|^α)" AND "automorphism groups in graphs"`

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

### 5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    d = 3

    # Generate a random d-regular graph with n vertices
    def generate_d_regular_graph(n, d):
        if (n * d) % 2 != 0 or d >= n:
            raise ValueError("Invalid parameters for generating a d-regular graph")

        adjacency_matrix = [[0] * n for _ in range(n)]
        edges_added = set()

        def add_edge(u, v):
            if u == v or (u, v) in edges_added or (v, u) in edges_added:
                return False
            adjacency_matrix[u][v] = 1
            adjacency_matrix[v][u] = 1
            edges_added.add((u, v))
            edges_added.add((v, u))
            return True

        for i in range(n):
            neighbors = random.sample(range(n), d)
            while not all(add_edge(i, j) for j in neighbors):
                neighbors = random.sample(range(n), d)

        return adjacency_matrix
```

```
graph = generate_d_regular_graph(n, d)

# Compute the order of the minimal automorphism group |A_G|
def is_automorphism(graph, perm):
    n = len(graph)
    for i in range(n):
        for j in range(i + 1, n):
            if (graph[i][j] != graph[perm[i]][perm[j]]):
                return False
    return True

def find_minimal_automorphism_group(graph):
    n = len(graph)
    identity = list(range(n))
    automorphisms = [identity]

    for perm in itertools.permutations(range(n)):
        if is_automorphism(graph, perm) and all(perm[i] != i for i in range(n)):
            automorphisms.append(perm)

    return automorphisms

A_G = find_minimal_automorphism_group(graph)
order_A_G = len(A_G)

# Compute the communication complexity rank variance σ(G)
def matrix_rank(matrix):
    m, n = len(matrix), len(matrix[0])
    rank = 0
    for i in range(m):
        if all(matrix[i][j] == 0 for j in range(n)):
            continue
        pivot_row = i
        for j in range(i + 1, m):
            if matrix[j][i] != 0:
                matrix[pivot_row], matrix[j] = matrix[j], matrix[pivot_row]
                break
        else:
            continue
        rank += 1
        for j in range(m):
            if j == pivot_row:
                continue
            factor = -matrix[j][i] / matrix[pivot_row][i]
            for k in range(n):
                matrix[j][k] += factor * matrix[pivot_row][k]
    return rank

sigma_G = matrix_rank(graph)

# Compute the correlation coefficient r^2 between |A_G| and σ(G)
def correlation_coefficient(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
```

```
        var_x = sum((x[i] - mean_x) ** 2 for i in range(n)) / n
        var_y = sum((y[i] - mean_y) ** 2 for i in range(n)) / n
        return cov_xy ** 2

    x = [order_A_G]
    y = [sigma_G]

    r_squared = correlation_coefficient(x, y)

    # Determine if the conjecture holds
    conjecture_holds = r_squared >= 0.9
    counterexample = "" if conjecture_holds else "correlation_coefficient < 0.9"

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": r_squared,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_r_squared = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_r_squared} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_r_squared} std=0.0 support_fraction={support_fraction
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.9\" first_failing_s
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: `INCONCLUSIVE`

**Reasoning**:

> The test timed out before producing data, which means it did not complete its execution to determine the correlation coefficient r^2 between |A_G| and σ(G). As a result, we cannot confirm whether the support condition of r^2 >= 0.9 is met. | next: Retry the experiment with increased time limits or optimize the code to ensure it completes within the given time frame.

## 11. Audit log (LLM calls)

**Total LLM calls**: 9

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 13499 |
| 2 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 9150 |
| 3 | novelty | ollama_remote | glm4:latest | 0 | 0 | 8164 |
| 4 | novelty | ollama_remote | glm4:latest | 0 | 0 | 18146 |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 44309 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 13476 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 16897 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 15296 |
| 9 | judge | ollama_remote | glm4:latest | 0 | 0 | 11851 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 150787 ms total latency. Provider mix: {'ollama_remote': 9}

*(full prompt+response transcripts available in `research/audit/4f9191200ded.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4f9191200ded.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/4f9191200ded.tar.gz` (if generated)